Clase 04 - Almacenamiento de datos

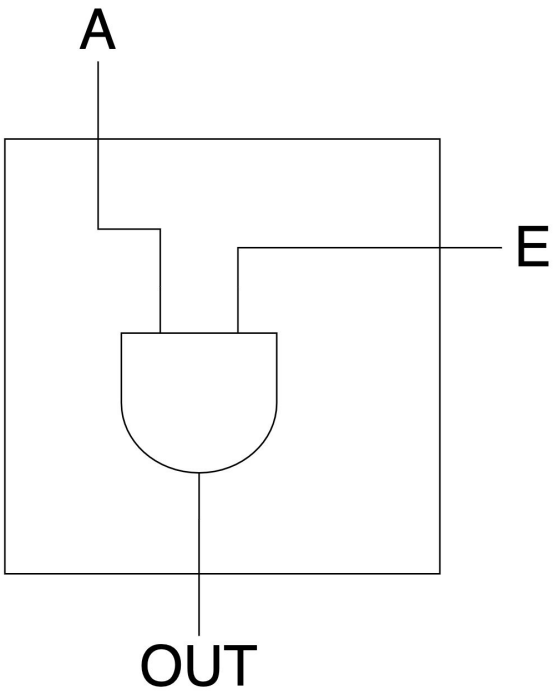
IIC2343 - Arquitectura de Computadores

Profesor:
- Felipe Valenzuela González
Correo:
frvalenzuela@alumni.uc.cl

Resumen de la clase pasada

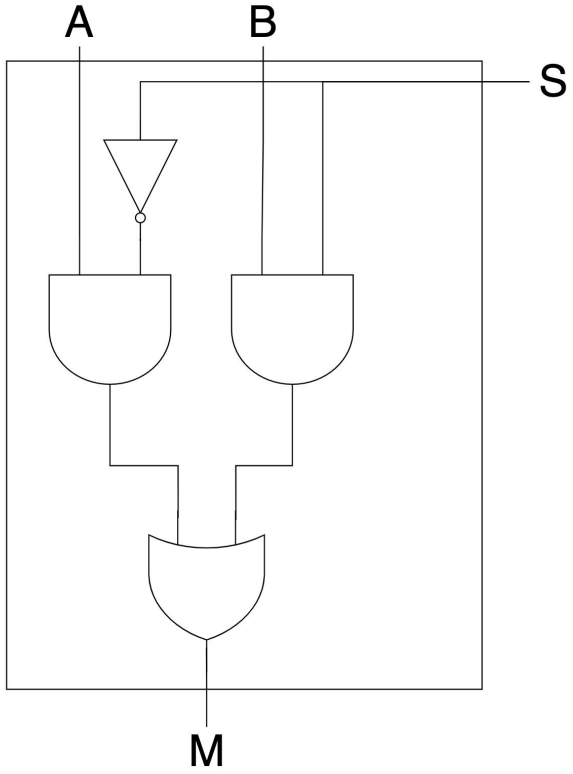
Circuitos de control - Enabler

OUT	E
0	0
A	1



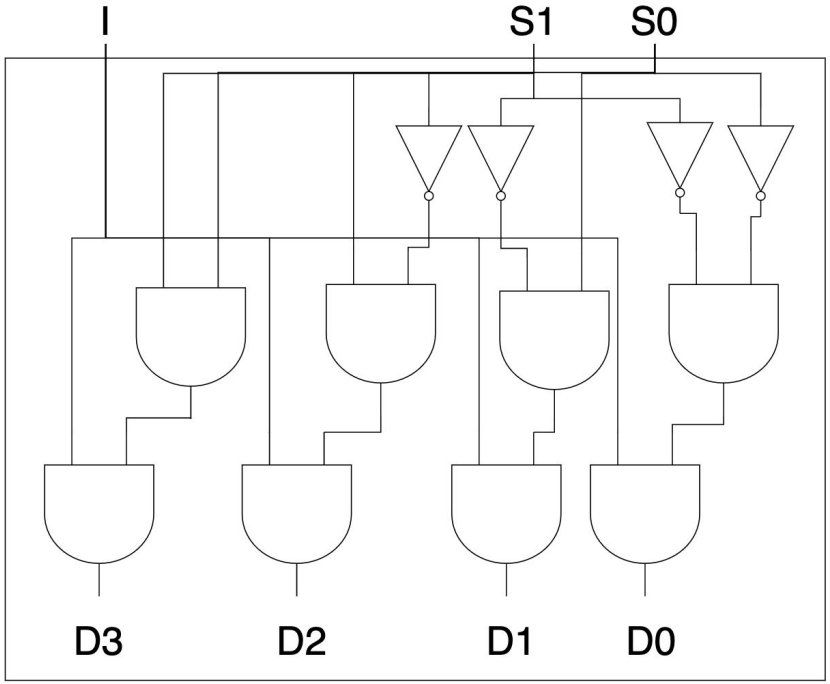
Circuitos de control - Multiplexor

S	M
0	A
1	B



Circuitos de control - Demux

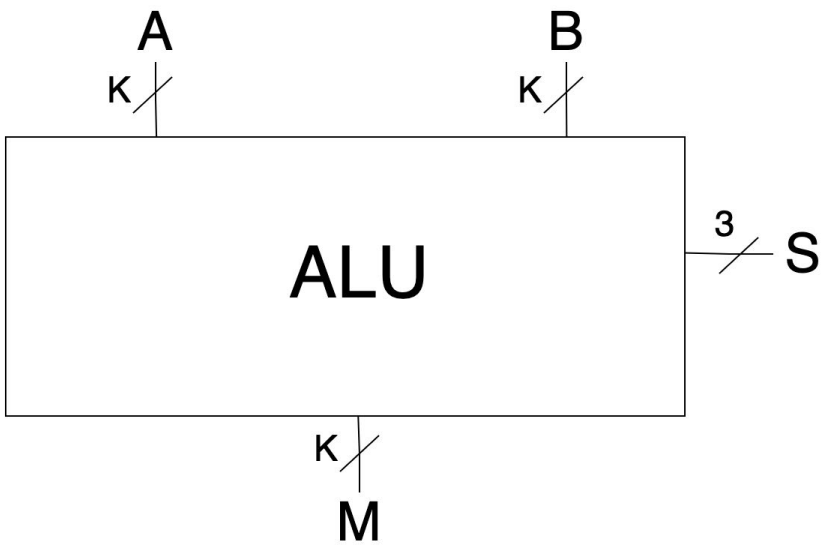
S1	S0	Output
0	0	D0 = I
0	1	D1 = I
1	0	D2 = I
1	1	D3 = I



Unidad Aritmética Lógica: ALU

- **Tabla de valores**:

S2	S1	S0	M
0	0	0	Suma
0	0	1	Resta
0	1	0	And
0	1	1	Or
1	0	0	Not
1	0	1	Xor
1	1	0	Shift left
1	1	1	Shift right

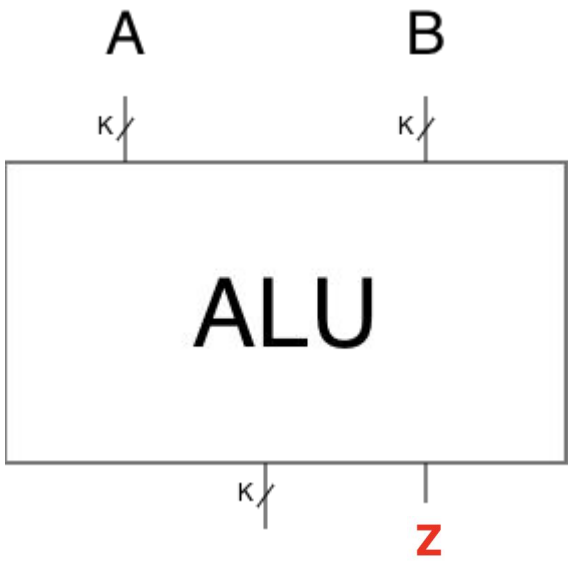


¿Dudas?

IMPORTANTE PARA LA ACTIVIDAD EVALUADA

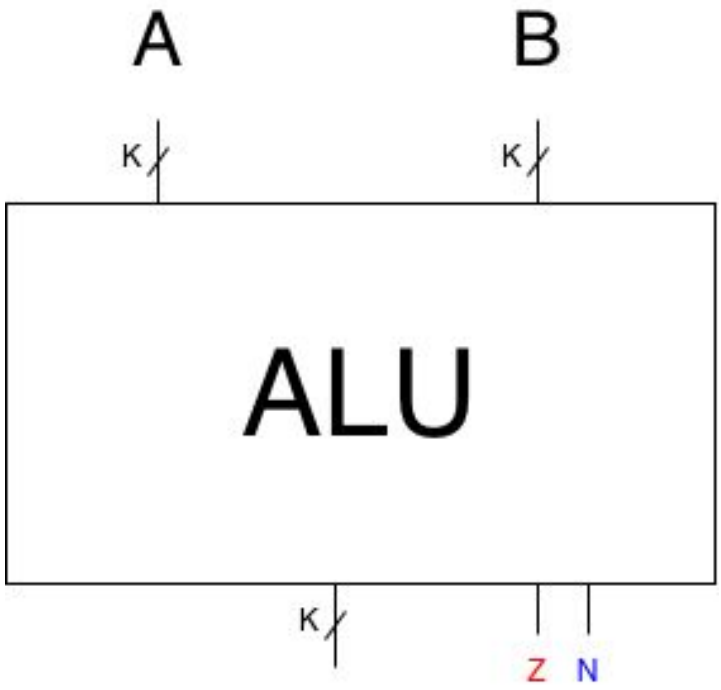
Unidad Aritmética Lógica: *Flags*

- De la ALU ahora obtenemos flags (señales de 1 bit) que nos indican el cumplimiento o no de una condición. Particularmente:
- **Z: Zero**. Indica si el resultado de la operación fue cero o no.



Unidad Aritmética Lógica: *Flags*

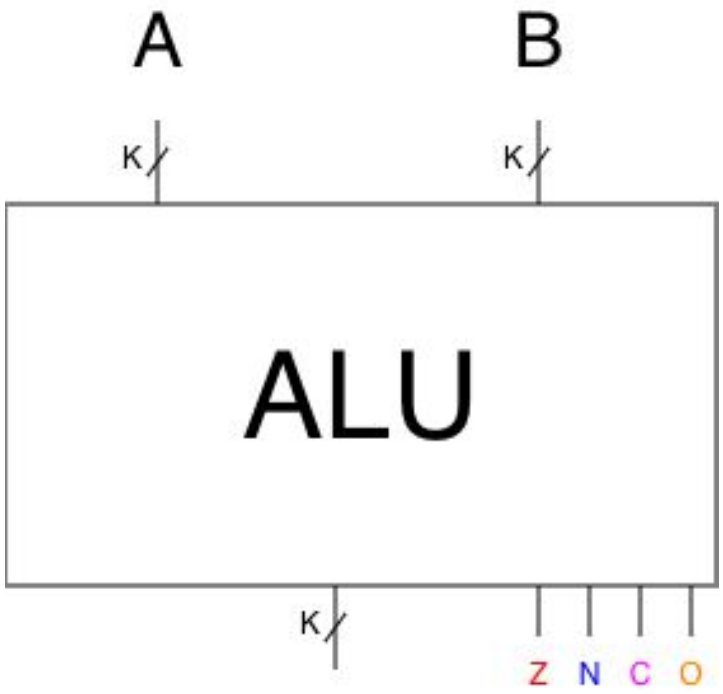
- De la ALU ahora obtenemos flags (señales de 1 bit) que nos indican el cumplimiento o no de una condición. Particularmente:
- **Z: *Zero***. Indica si el resultado de la operación fue cero o no.
- **N: *Negative*** Indica si el resultado de la operación fue negativo o no



Unidad Aritmética Lógica: *Flags*

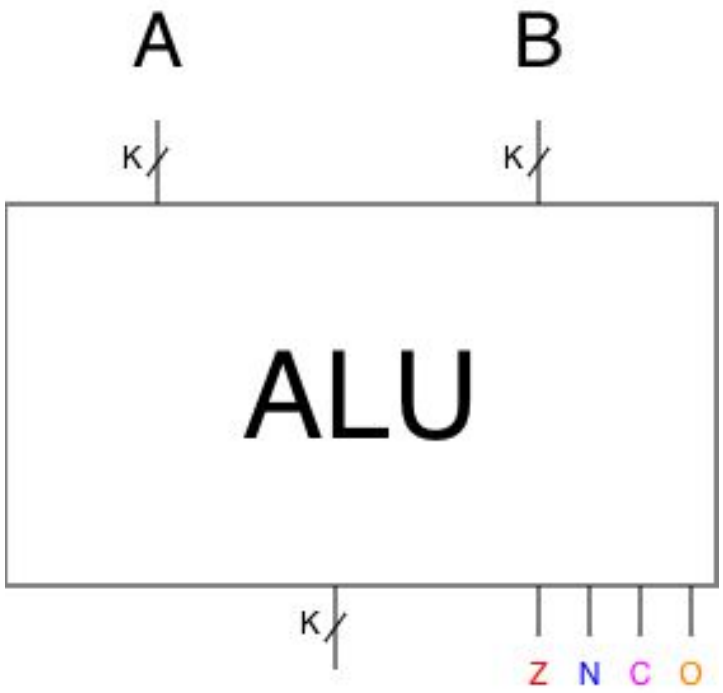
- **C**: ***Carry***. Indica si hubo un bit de carry al finalizar la operación.
- **O**: ***Overflow***. Indica si hubo o no overflow al finalizar la operación.

- Ahondaremos un poco más en estas nuevas flags.



Unidad Aritmética Lógica: *Flags* - C y O

- **Flag C:** Es 1 si hubo bit de carry en una operación ADD/SUB, pero también se utiliza para representar el bit "descartado" en operaciones tipo shift.
  - **Ejemplo: SHR(00101011) → C = 1**
- **Flag O:** Es 1 si hubo overflow en operaciones ADD/SUB. Algunas implementaciones también lo consideran en operaciones tipo shift, pero no todas.
- Existen cuatro casos a considerar**:**
  - **$A \geq 0$; $B \geq 0 \rightarrow O = 1$ si $A + B < 0$**
  - **$A < 0$; $B < 0 \rightarrow O = 1$ si $A + B \geq 0$**
  - **$A \geq 0$; $B < 0 \rightarrow O = 1$ si $A - B < 0$**
  - **$A < 0$; $B \geq 0 \rightarrow O = 1$ si $A - B \geq 0$**



¿Dudas?

Introducción del curso:

- Un computador lo definimos como una **máquina programable que ejecuta programas.**
- Para programar necesitamos:
    - Datos: números (enteros, reales) , texto, imágenes, etc ✓
    - Operaciones: suma, resta, multiplicación, división, etc ✓
    - Variables: simples, arreglos ?
    - Control de flujo: comparaciones, manejo de ciclos
- Hoy veremos **lo básico para tener variables**

Almacenamiento de datos

- Es componente esencial de todo computador
- Es *hardware* especializado para **almacenar** un estado
- Permite realizar **un cambio** en un **instante determinado de tiempo**

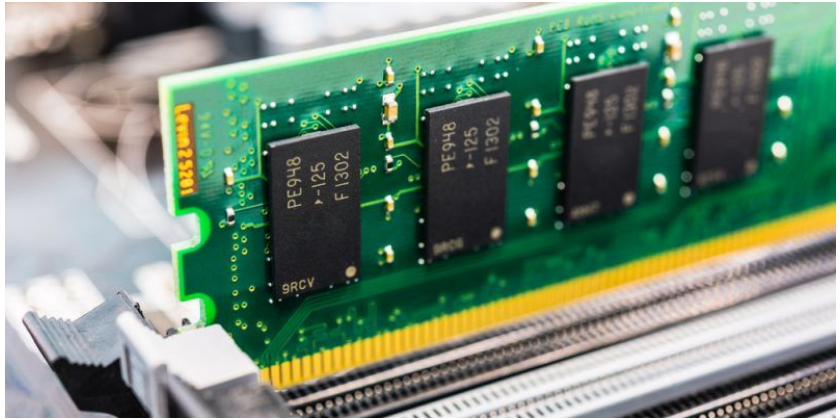
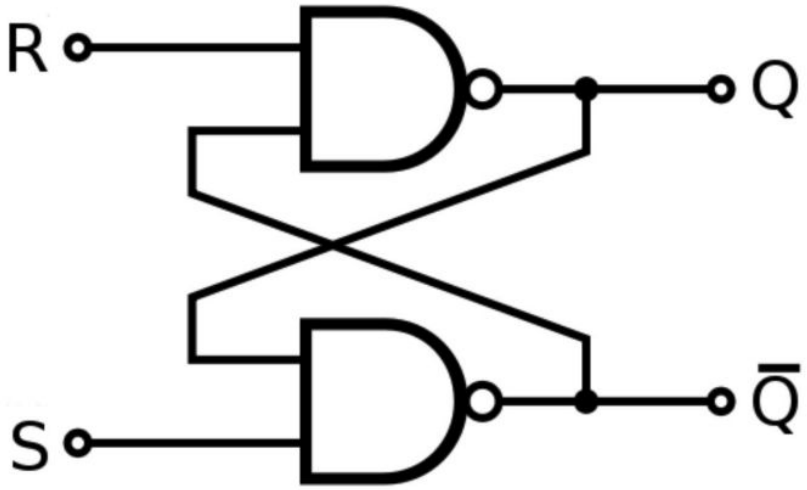


Imagen Referencia: https://concepto.de/memoria-ram/

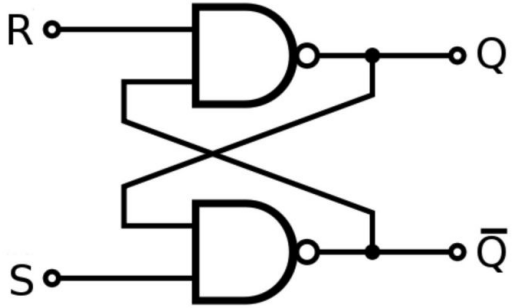
Almacenamiento de datos: Latch RS

S	R	Q(t+1)
0	0	-
0	1	0
1	0	1
1	1	Q(t)



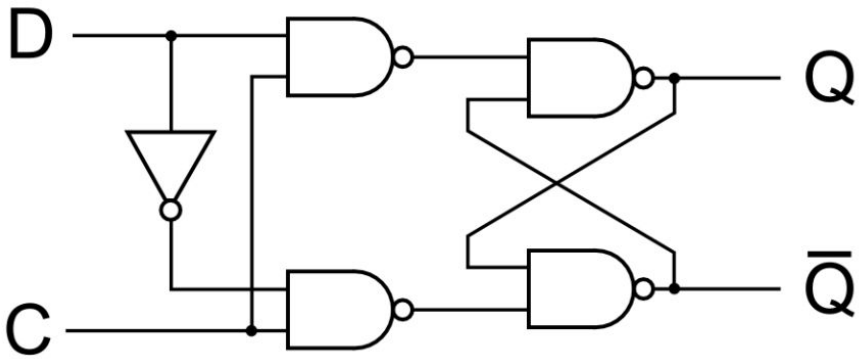
Almacenamiento de datos: Latch RS

S	R	Q(t+1)
0	0	-
0	1	0
1	0	1
1	1	Q(t)



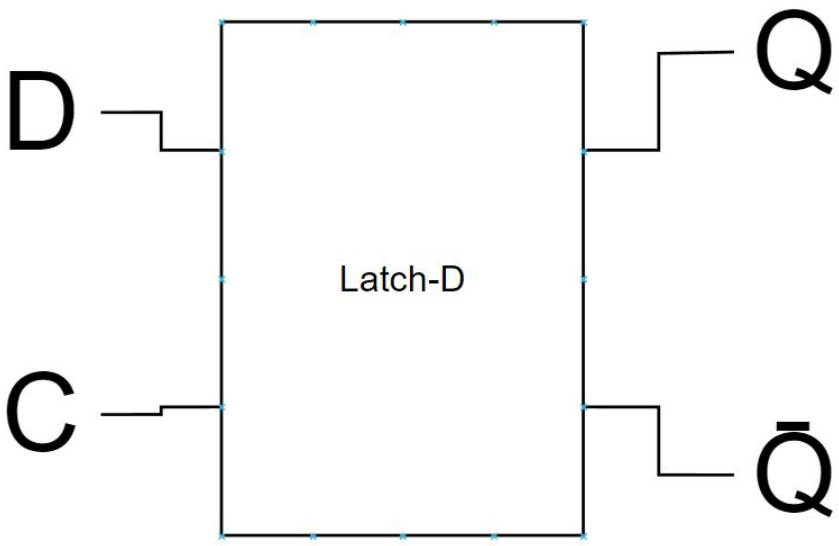
Almacenamiento de datos: Latch D

C	D	Q(t+1)
0	0	Q(t)
0	1	Q(t)
1	0	0
1	1	1



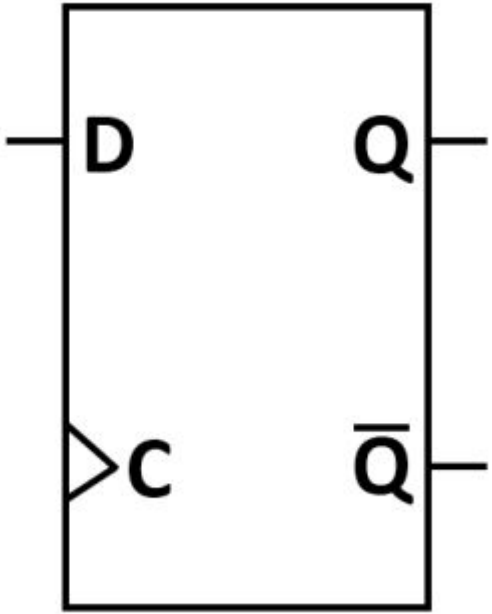
Almacenamiento de datos: Latch D

- D: Dato
- C: control
- Q: estado
- Esta es una abstracción usada para denotar el latch D



Almacenamiento de datos: Latch D

- Los **latches** se activan en un estado (1 ó 0), lo que no soluciona del todo nuestros problemas.
- Necesitamos una pieza similar, pero que se active sólo en **un instante dado**.



Almacenamiento de datos: Relojes

- Un reloj —en un circuito digital— es simplemente un circuito que emite una serie de pulsos, con dos propiedades:
- El ancho de pulso es preciso
- El intervalo entre pulsos consecutivos es preciso

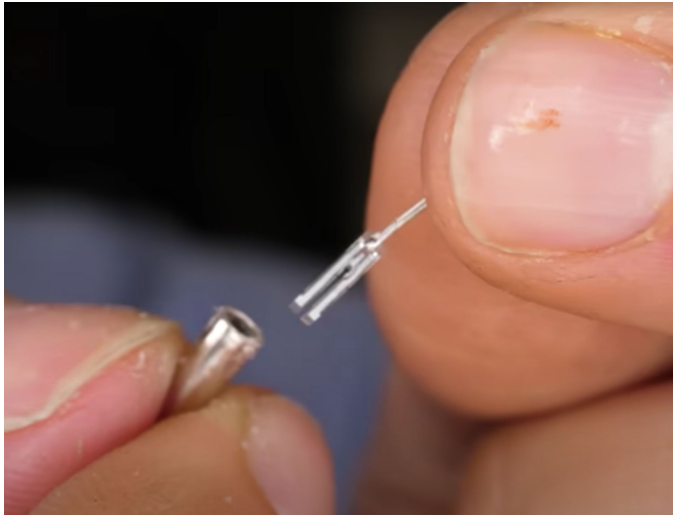
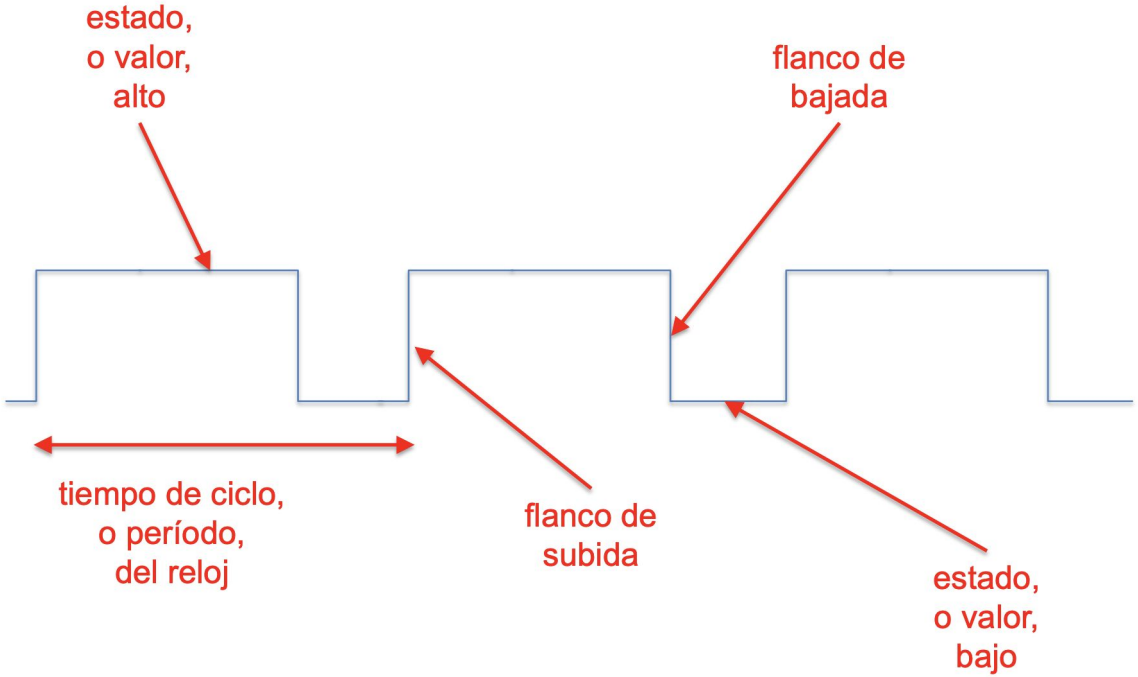


Imagen Referencia: https://www.youtube.com/watch?v=_2By2ane2I4

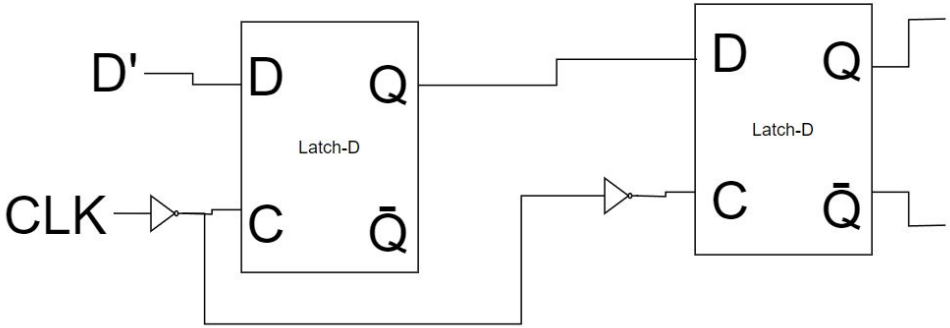
Almacenamiento de datos: Relojes



¿Dudas?

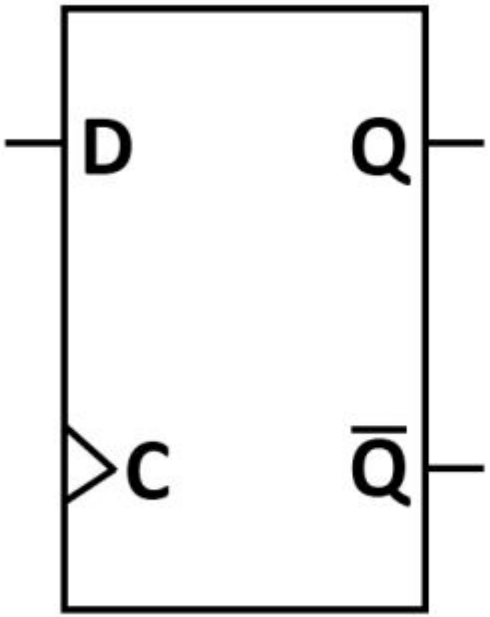
Almacenamiento de datos: Flip-Flop D

C	D	Q(t+1)
↑	0	0
↑	1	1
*	*	Q(t)



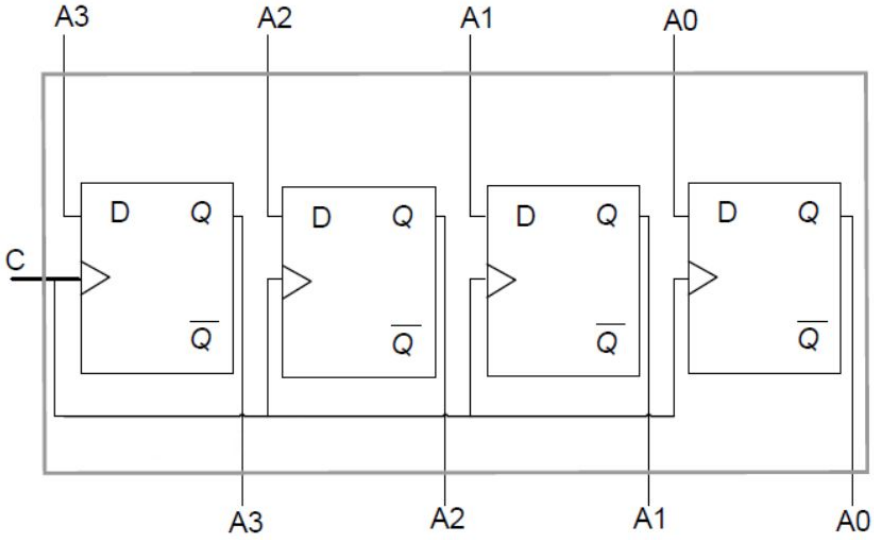
Almacenamiento de datos: Registro

- Los **flip-flop** se activan solo en una transición o flanco (subida o bajada).
- Cuando una componente tenga un input con símbolo un triángulo, significa que ese input es un clock.



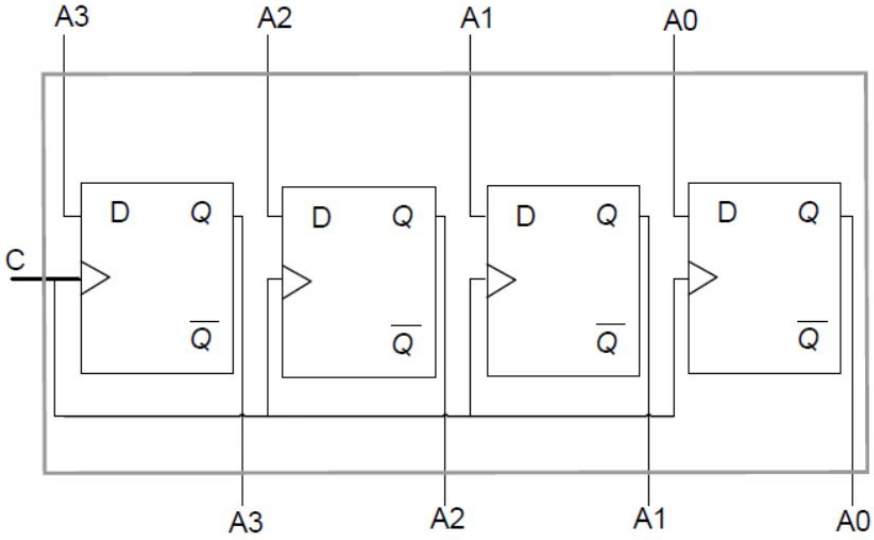
Almacenamiento de datos: Registros

- Llamaremos registros al conjunto de flip-flops
- Si queremos almacenar *N* bits necesitaremos *N* flip-flops



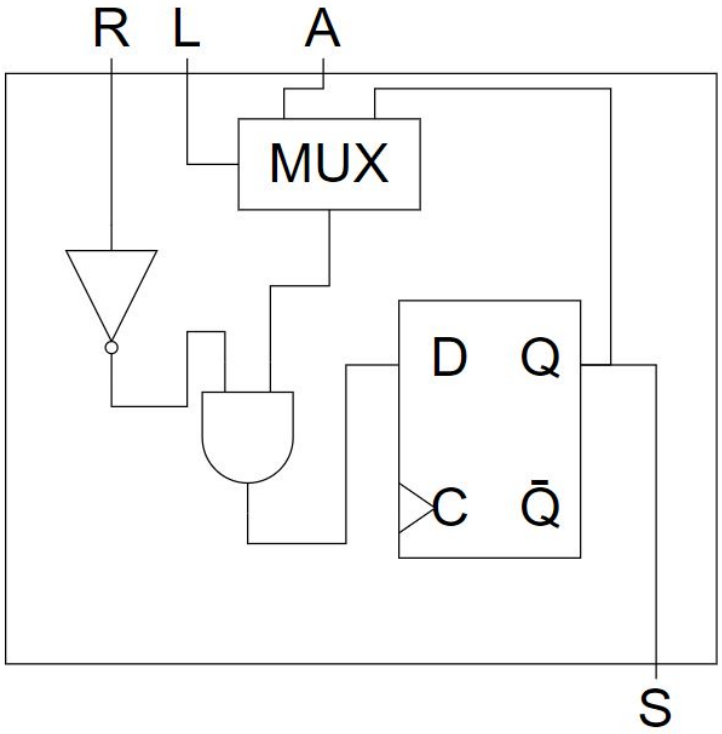
Almacenamiento de datos: Registros

- Llamaremos registros al conjunto de flip-flops
- Si queremos almacenar *N* bits necesitaremos *N* flip-flops
- Nos **falta control de almacenamiento**



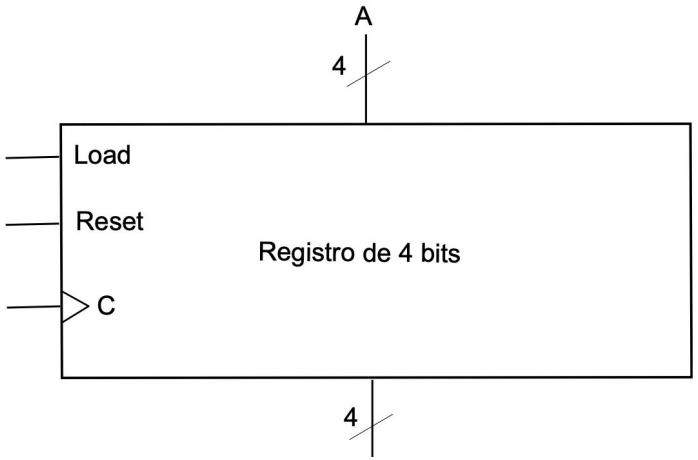
Almacenamiento de datos: Registros

C	L	R	S
↑	1	0	A
↑	*	1	0
*	0	0	Q(t)



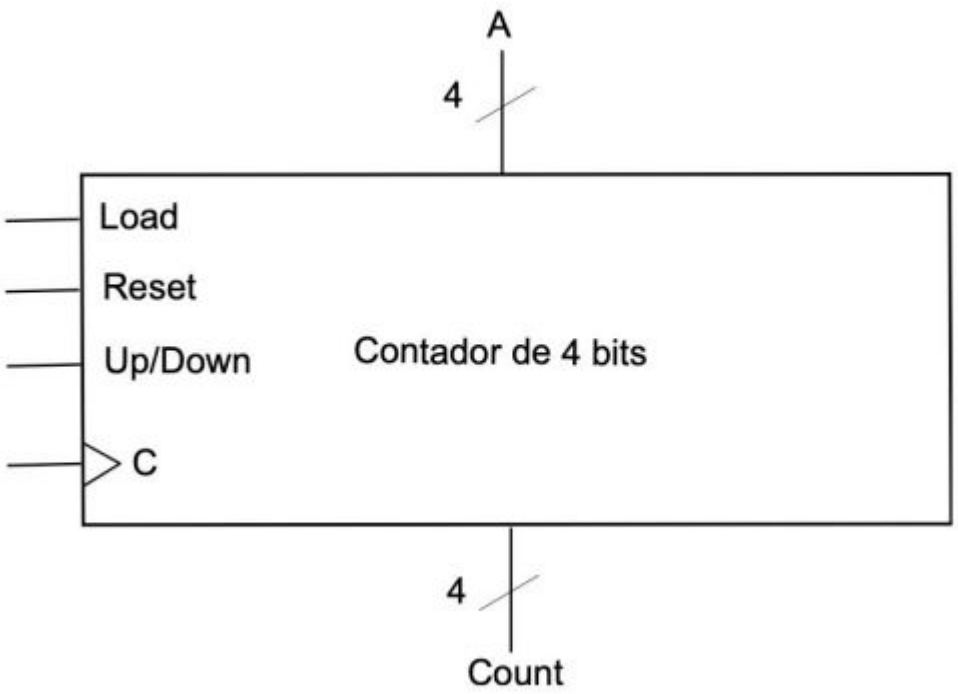
Almacenamiento de datos: Registros

- El registro tradicional tiene la función de carga y reset
- Se extiende con el manejo de varios flip-flops
- Nos permite almacenar unidades de información o **palabras**



Almacenamiento de datos: Registro Contador

- Con lo anterior se puede generalizar para tener un registro contador
- Una señal Up indicando que el valor almacenado se suma uno
- Una señal Down indicando que el valor almacenado se resta uno



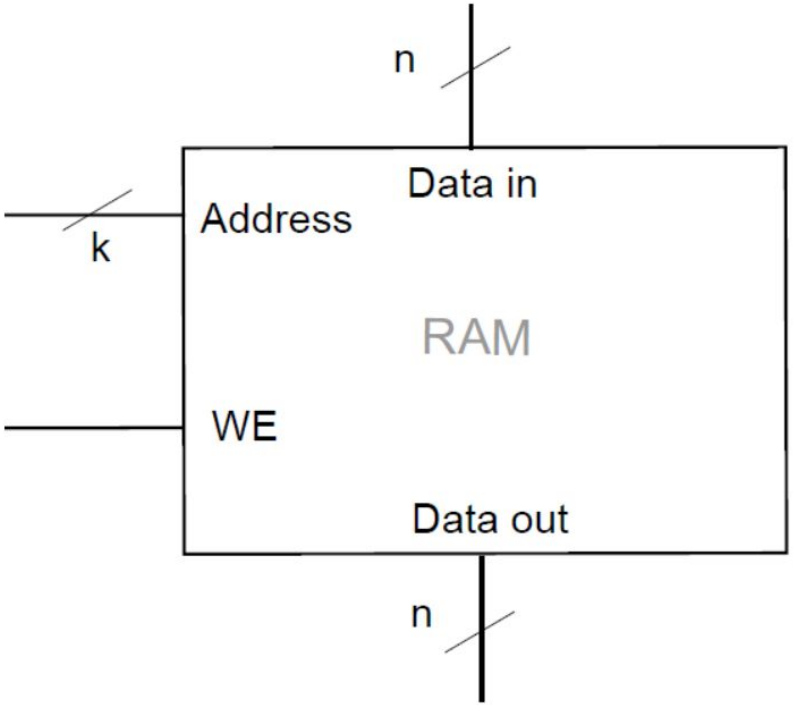
¿Dudas?

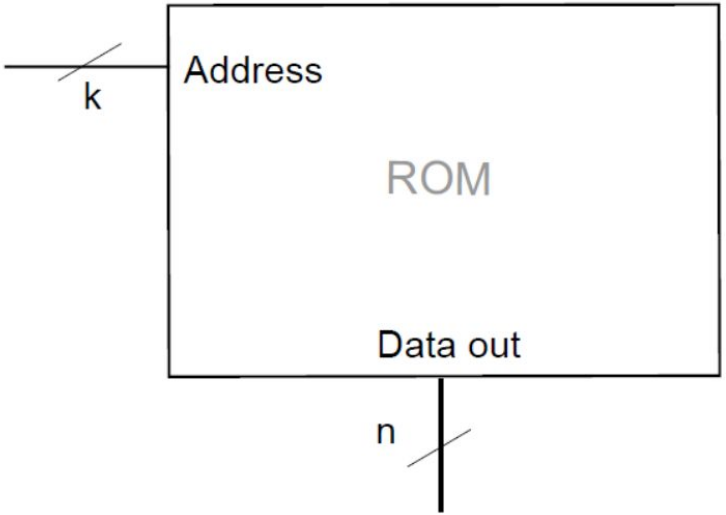
Almacenamiento de datos: Memorias

- Componentes que permite acceder, y en algunos casos, modificar distintos datos según una **dirección de memoria**
- El proceso de acceso se conoce como **direccionamiento**

Dirección en decimal	Dirección en binario	Palabra
0	000	$Palabra_0$
1	001	$Palabra_1$
2	010	$Palabra_2$
3	011	$Palabra_3$
4	100	$Palabra_4$
5	101	$Palabra_5$
6	110	$Palabra_6$
7	111	$Palabra_7$

Almacenamiento de datos: Memorias





Almacenamiento de datos: Variable

- Valores temporales que se actualizan según flujo del código
- Se almacenan en distintos componentes, como puede ser registros o RAM
- Todo variable tiene un único **tipo de dato**

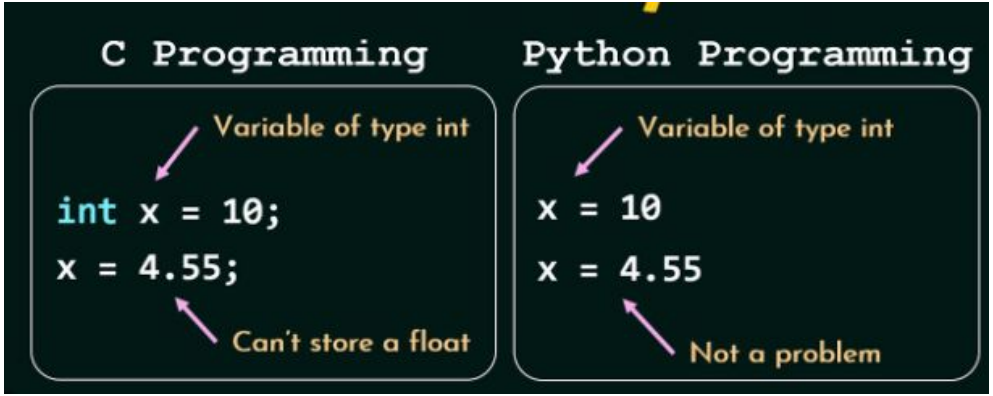


Imagen Referencia: https://www.youtube.com/watch?app=desktop&v=X6TcB0DNLE8

Almacenamiento de datos: Endianness

Address	Value
00	0x0F
01	0xA7
10	0x1D
11	0x23

- int 32 bits big endian:

  0x 0F A7 1D 23

- int 32 bits little endian:

  0x 23 1D A7 0F

Almacenamiento de datos: Arreglos

- Es un caso especial de variable
- Se compone de cuatro atributos: **tipo de dato**, **endianness**, **largo** y **dirección de inicio**
- Se puede **indexar** cada elemento para acceder a ellos en nuestra memoria

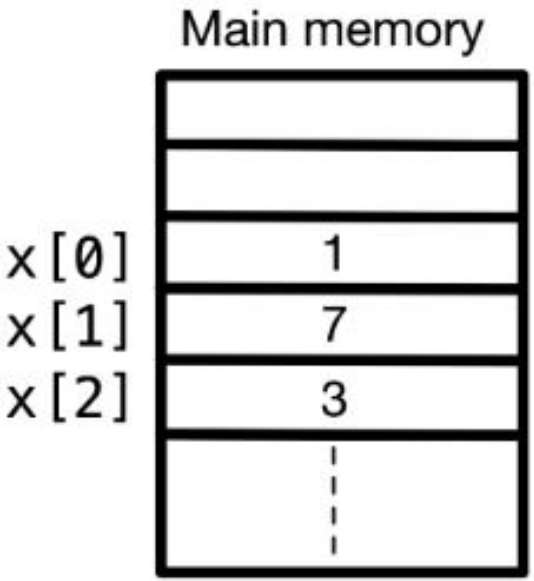
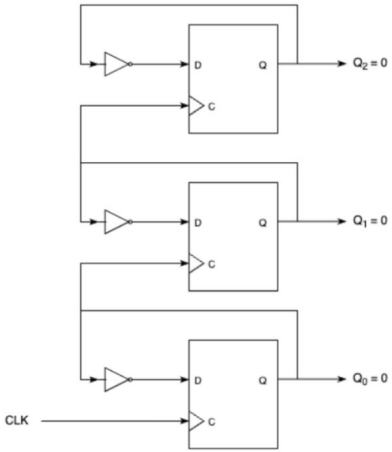


Imagen Referencia: https://learningc.org/chapters/chapter07-arrays/what-are-arrays

¿Dudas?

Ejercicio prueba I1 - 2024-1

(c) (3 ptos.) Suponga que posee un componente con una salida de 3 bits igual a $Q_2Q_1Q_0$ conectado a una señal de control `CLK`. Su circuito interno se compone de flip-flops de tipo D conectados de la siguiente forma:



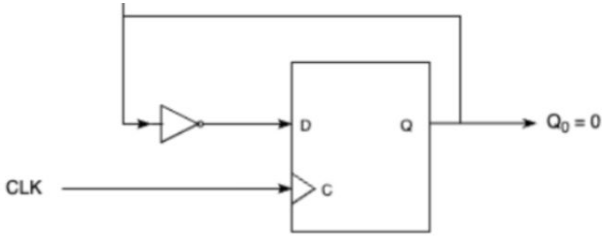
Asumiendo que se tiene un valor inicial $Q_2Q_1Q_0 = 000$, indique, justificadamente, cómo se va modificando el estado del componente con cada flanco de subida de la señal `CLK`.

Ejercicio prueba I1 - 2024-1

C	D	Q(t+1)
↑	0	0
↑	1	1
*	*	Q(t)

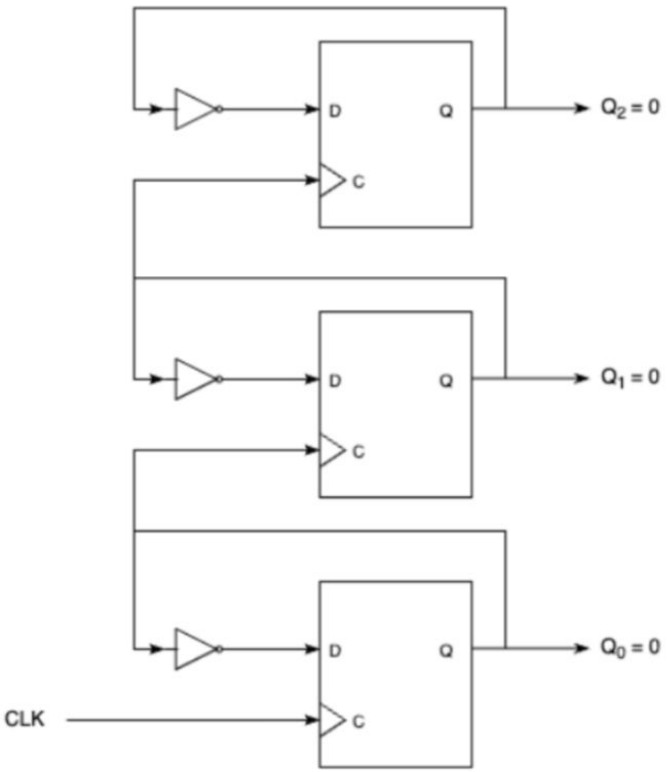
- Es D es el dato que quiero guardar.
- Q es el estado (lo que tengo guardado).
- Solo cuando ocurre un flanco de subida del reloj (↑), el flip-flop copia D en Q.
- En cualquier otro momento, Q se queda igual

Ejercicio prueba I1 - 2024-1



- Si al flip-flop D le conecto como entrada el valor contrario de su salida (D = NOT Q):
- En cada flanco de subida del reloj, el flip-flop guarda lo opuesto de lo que tenía.
- Ejemplo: si Q=0, en el próximo flanco sube a 1; en el siguiente, baja a 0; después sube a 1, etc.
- Eso significa que la salida alterna: 0 → 1 → 0 → 1… cada vez que llega un pulso de reloj.
- Este es el “**truco**” para que cada flip-flop cambie de estado de manera automática.

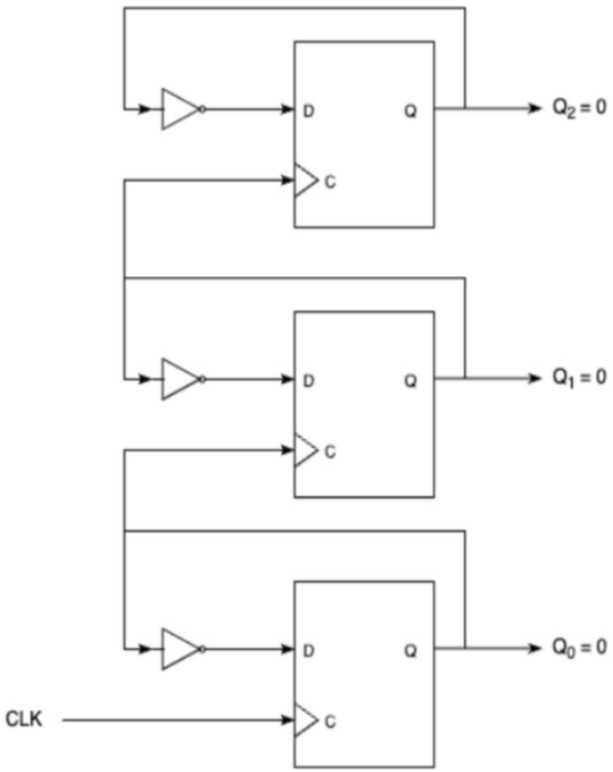
Ejercicio prueba I1 - 2024-1



- Hay tres flip-flops D que generan las salidas Q2, Q1 y Q0.
- El primero (Q0) recibe el reloj global.
- El segundo (Q1) no recibe el reloj global, sino que usa Q0 como su reloj.
- El tercero (Q2) usa Q1 como su reloj.
- Eso significa que:

- Q0 cambia en cada flanco del reloj global.
- Q1 cambia cuando Q0 pasa de 0 a 1.
- Q2 cambia cuando Q1 pasa de 0 a 1

Ejercicio prueba I1 - 2024-1

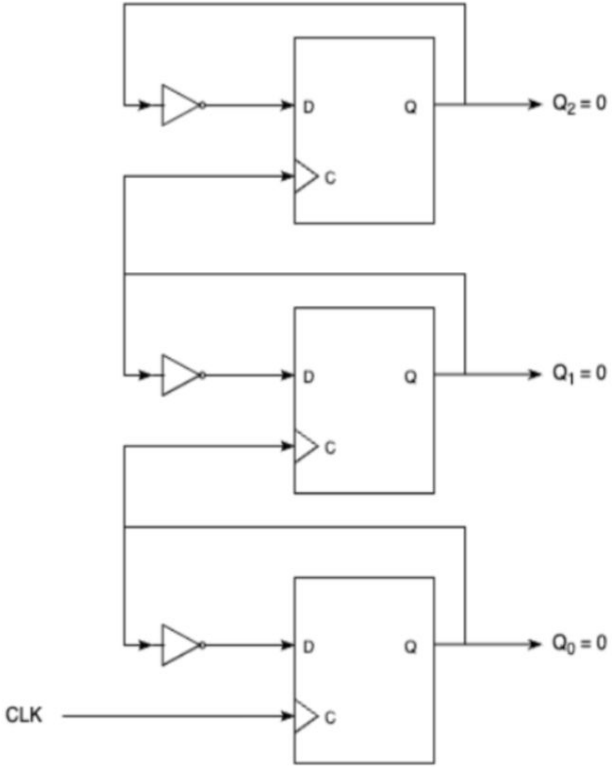


Evolución del estado (paso a paso)

Partimos en **Q2Q1Q0 = 000**. Veamos qué pasa en cada flanco del reloj global:

1. **000 → 111**
   - Reloj sube → Q0: 0→1.
   - Ese cambio de Q0 hace que Q1 también cambie: 0→1.
   - Ese cambio de Q1 hace que Q2 también cambie: 0→1.
2. **111 → 110**
   - Q0: 1→0.
   - Como fue bajada, Q1 no cambia → sigue en 1.
   - Q2 sigue en 1.
3. **110 → 101**
   - Q0: 0→1.
   - Ahora Q1 cambia: 1→0.
   - Como Q1 bajó, Q2 no cambia → queda en 1.

Ejercicio prueba I1 - 2024-1

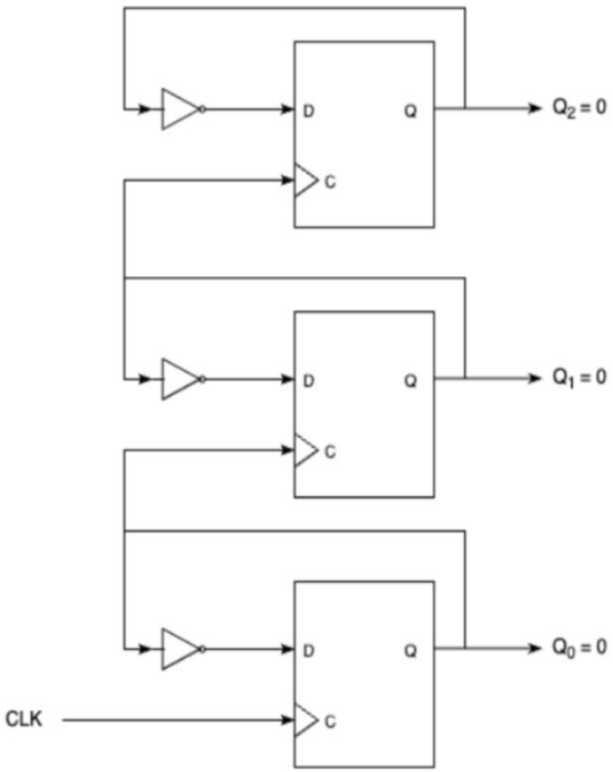


Evolución del estado (paso a paso)

Partimos en **Q2Q1Q0 = 000**. Veamos qué pasa en cada flanco del reloj global:

4. **101 → 100**
   - Q0: 1→0.
   - Q1 no cambia.
5. **100 → 011**
   - Q0: 0→1.
   - Q1: 0→1.
   - Q2: 1→0.
6. **011 → 010**
   - Q0: 1→0.
   - Q1 no cambia.

Ejercicio prueba I1 - 2024-1



Evolución del estado (paso a paso)

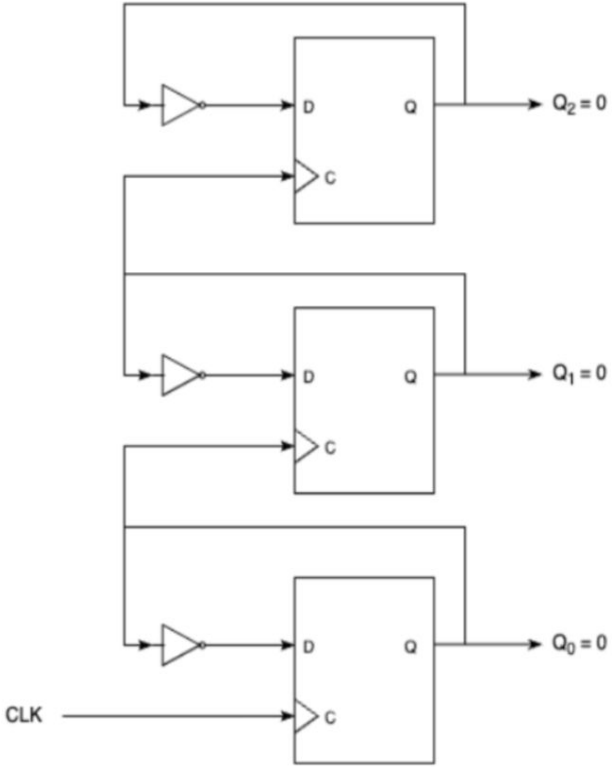
7. **010 → 001**
   - Q0: 0→1.
   - Q1: 1→0.
   - Q2 no cambia.
8. **001 → 000**
   - Q0: 1→0.
   - Q1 y Q2 no cambian.

Y volvemos al inicio. La secuencia completa es:

000 → 111 → 110 → 101 → 100 → 011 → 010 → 001 → 000.

-

Ejercicio prueba I1 - 2024-1



Si miramos los estados como números binarios:

- 000 = 0
- 111 = 7
- 110 = 6
- 101 = 5
- 100 = 4
- 011 = 3
- 010 = 2
- 001 = 1
- 000 = 0 (reinicio)
  La salida va **bajando de a uno en binario**. Es decir: este circuito es **un contador de 3 bits que va decrementando en cada pulso de reloj**

¿Dudas?

Bibliografía

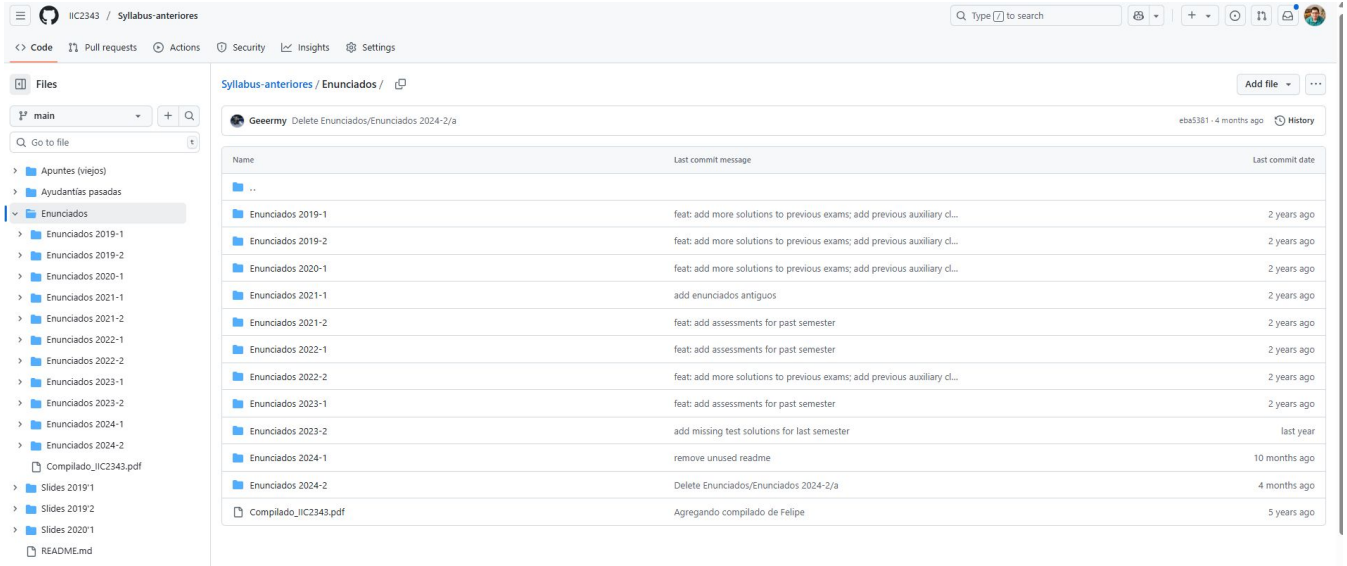
- Apuntes históricos. Hans Löbel, Alejandro Echeverría

  04 - Almacenamiento de datos

- D. Patterson, Computer Organization and Design RISC-V. Edition: The Hardware Software Interface. Morgan Kaufmann, 2020.Capítulos A.7-A.8-A.9. Página A-47, 792 en PDF

Bibliografía

- https://github.com/IIC2343/Syllabus-anteriores/tree/main/Enunciados



Introducción del curso:

- Un computador lo definimos como una **máquina programable que ejecuta programas.**
- Para programar necesitamos:
    - Datos: números (enteros, reales) , texto, imágenes, etc ✓
    - Operaciones: suma, resta, multiplicación, división, etc ✓
    - Variables: simples, arreglos ✓
    - Control de flujo: comparaciones, manejo de ciclos ?
- Para poder tener esta última parte requerimos de comenzar el capítulo de **¡programabilidad!**

Clase 04 - Almacenamiento de datos

IIC2343 - Arquitectura de Computadores

Profesor:
- Felipe Valenzuela González
Correo:
frvalenzuela@alumni.uc.cl